

INTERNSHIP TECHNICAL DESIGN REPORT

SDK Product CRUD

A proof of modular SalesBuzz page integration

CORE CLAIM

A business page can be developed separately and integrated into SalesBuzz by reusing BI frontend components and backend platform services.

Prepared for: Internship evaluation

Project type: Proof of concept / reference implementation

Date: 30 August 2026

SECTION 01

Executive summary

What was built, why it matters, and what the trial is designed to prove.

The Product page is the concrete deliverable. Its broader value is the reusable boundary it exposes between a page-specific workflow and the shared SalesBuzz/BI platform contract.

This proof of concept combines an Angular 20 Product Management page, an ASP.NET Core API, BI-SDK 0.2.0, OData, Entity Framework Core, and SQL Server. It supports secure login, Product search and paging, summary metrics, create/update/delete operations, role-based permissions, and business-unit isolation.

The technical result is more important than a single CRUD screen: the Product page consumes shipped BI frontend packages and participates in the same backend authentication, permission, BUID, query, and persistence conventions. This is evidence that another separately developed business page can follow the same integration pattern.

Trial objective

- Demonstrate a realistic business page built on reusable BI and SalesBuzz foundations.
- Keep Product-specific behavior separate from shared UI, security, and data-access services.
- Exercise the integration seam from the custom page through BI packages and SDK services to SQL Server.
- Identify the remaining platform work needed to load and govern pages inside production SalesBuzz.

IN SCOPE	OUT OF SCOPE
Product CRUD, summaries, OData, JWT, permissions, BUID isolation, BI Grid, BI Navigation, PublicApiClient, SQL schema, and verification.	Dynamic plug-in installation, automatic page discovery, production-shell loading, navigation registration, deployment governance, and long-term operations.

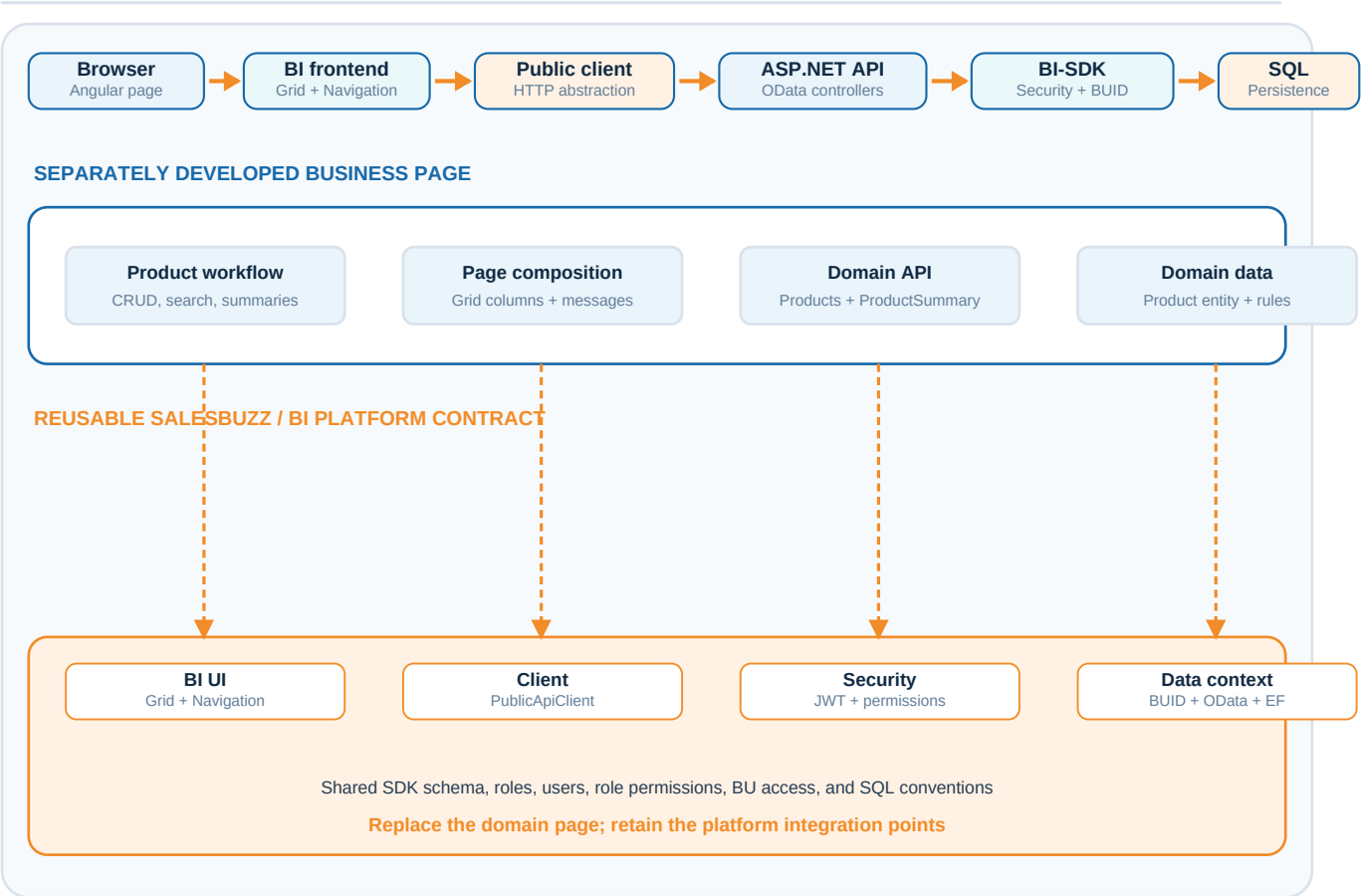
Evaluation lens

The report distinguishes three levels of evidence: implemented behavior in the repository, verified behavior from the available test environment, and production recommendations that are not yet implemented. This avoids treating a successful trial as a finished extension platform.

SECTION 02

System architecture

A page-specific Product layer connected to reusable BI and SalesBuzz capabilities.



The browser hosts an Angular standalone application. BI Grid and BI Navigation provide the core data-management experience; ProductDataSource adapts the Product domain to their data-source contract. PublicApiClient carries relative requests to the ASP.NET Core API. The API registers the BI-SDK services for JWT validation, permissions, current business unit, OData, exception handling, and the shared database base context.

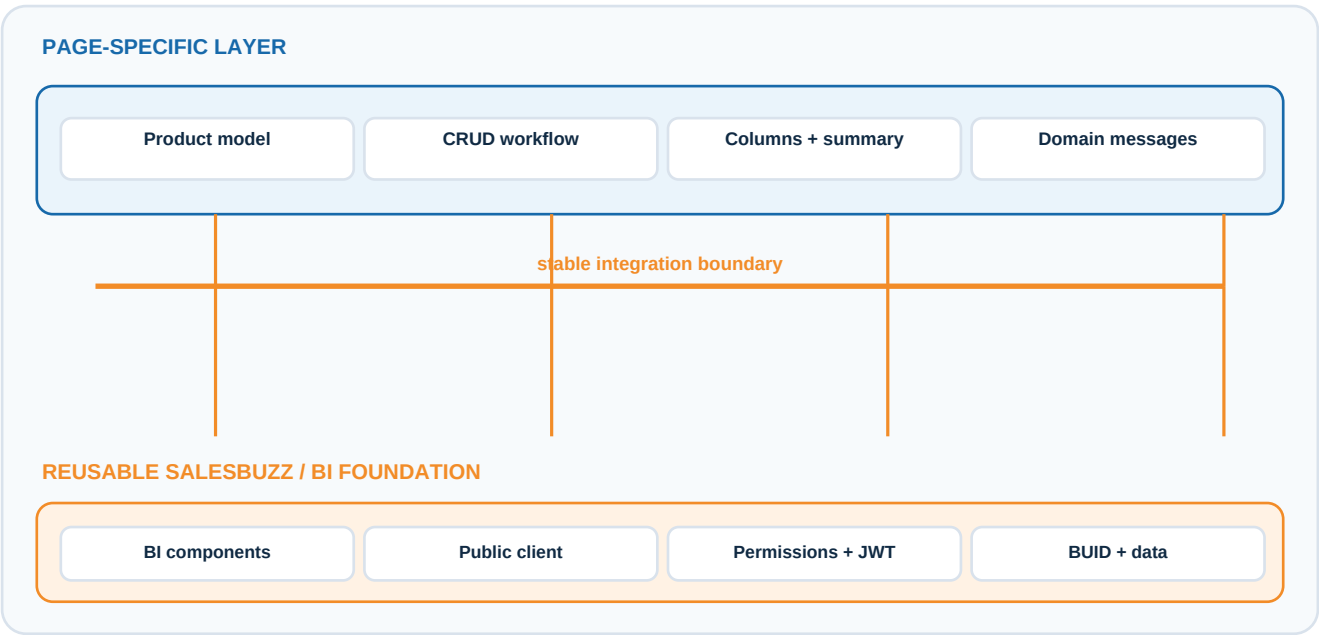
Technology stack

Layer	Technology	Role in the trial
Page	Angular 20 + TypeScript	Composes login, summaries, Product search, BI Navigation, and BI Grid.
Reusable UI	bi-modules + bi-interfaces + Kendo	Provides consistent grid, navigation, paging, editing, and toolbar behavior.
Client contract	@salesbuzz/public-sdk	Abstracts frontend API access so domain services do not depend directly on HttpClient.
Backend	ASP.NET Core / .NET 10 + BI-SDK 0.2.0	Hosts login, Product OData, summary, JWT, permission, and current-BU integration.
Data	EF Core + SQL Server 2022	Maps Product data while preserving the SDK security and business-unit schema.

SECTION 03

Modularity feasibility

How the working page demonstrates a repeatable SalesBuzz extension boundary.



Replace the Product-specific domain page; retain the same frontend components and backend platform integration points.

Evidence, not aspiration

Claim	Implemented evidence	Why it supports modularity
Shared frontend	The page imports BIModulesModule and renders BI-Nav and BI-Grid from shipped packages.	A new page can reuse the same visual and interaction conventions.
Shared client	Product services call relative URLs through PublicApiClient.	The domain page depends on a stable platform request abstraction.
Shared security	BI-SDK JWT registration and HasPermission protect API operations.	A page participates in the platform identity and permission model.
Shared tenant context	ICurrentBUContext supplies BUID for reads and writes.	Customer data scoping is reusable rather than recreated per page.
Shared data conventions	OData, SalesBuzzDbContextBase, EF Core, and SDK SQL tables are retained.	The page adds domain data without replacing the platform data contract.

What is page-specific

The Product entity, columns, summaries, route names, messages, and CRUD workflow remain clearly identifiable. This matters because a useful module boundary must let a team change domain behavior without changing shared platform services, and let BI evolve internal platform implementations without rewriting every business page.

SECTION 04

Component design

Responsibilities remain focused so the page and platform can evolve independently.

Component	Responsibility	Depends on
App	Composes login state, identity, summaries, search, navigation, and grid.	AuthService, ProductDataSource, ProductSummaryService, BI components
ProductDataSource	Adapts OData and CRUD responses to the BI Grid contract.	PublicApiClient, Product columns
AuthService + interceptor	Stores a short-lived session and attaches JWTs to protected Product requests.	PublicApiClient, sessionStorage
AuthController	Validates active users and password hashes, reads Product permissions, and issues JWTs.	SDK users/roles, AppCredentials, JWT configuration
ProductsController	Implements OData CRUD with permission and current-BU enforcement.	CatalogDbContext, HasPermission, ICurrentBUContext
ProductSummaryController	Aggregates BU-scoped Product metrics.	CatalogDbContext, HasPermission, ICurrentBUContext
CatalogDbContext	Maps Product and trial credentials on top of SalesBuzzDbContextBase.	BI-SDK EF Core integration, SQL Server

Frontend composition

- BI Navigation receives the logged-in user's create, update, and delete flags; Viewer therefore sees a read-only toolbar.
- BI Grid receives Product columns, ProductDataSource, OData paging, and Product change tracking.
- Search creates an escaped OData `contains(Name, ...)` filter with count and paging parameters.
- Summary cards show Product count, inventory value, active count, and low-stock count for the current BU.

Backend composition

- Program registers SalesBuzz OData, exception handling, JWT validation, authorization, current-BU services, and catalog data.
- Middleware order establishes the static request context before authentication and authorization.
- The OData Product set remains page-specific while permission and BUID behavior come from shared SDK services.

Design principle: each business page owns its workflow; BI owns the stable capabilities every page should inherit.

SECTION 05

Security and business-unit isolation

The reusable platform contract is enforced in the API, not only reflected in the UI.



Backend checks remain authoritative even when UI actions are hidden
Cross-BU keys return 404; writes receive the authenticated BUID

Account model	Business unit	Read	Create	Update	Delete
Admin	C100	Yes	Yes	Yes	Yes
Viewer	C200	Yes	No	No	No

Authentication contract

POST /Auth/Login verifies an active SDK user and a stored ASP.NET Core password hash, reads that role's Products permission record, and issues a 15-minute HMAC-SHA256 JWT. The token contains the exact Name, Role, BUID, and JTI claims used by the integration. Issuer, audience, signing key, and trial passwords come from local configuration; plaintext passwords are not persisted in SQL.

Authorization and isolation rules

- GET requires Products Read and filters the query by authenticated BUID before OData applies count, paging, sorting, or search.
- POST requires Create and overwrites any client-supplied BUID with the authenticated BUID.
- PATCH requires Update, finds the Product by key plus BUID, and forbids changes to Id or BUID.
- DELETE requires Delete and finds the Product by key plus BUID.
- Cross-BU keys return 404, avoiding disclosure that another business unit owns the record.

The toolbar improves the user experience. HasPermission and current-BU filtering remain the actual security boundary for direct API requests.

SECTION 06

Data model and request flows

A compact domain model built on the shared SDK security and business-unit schema.

Field	SQL mapping	Rules	Ownership
Id	int identity / PK	Server generated; patch protected	Product API
Name	nvarchar(100)	Required	Product page/API
Price	decimal(18,2)	Required; at least 0	Product page/API
StockQuantity	int	Required; at least 0	Product page/API
IsActive	bit	Required; defaults true	Product page/API
BUID	nvarchar(15)	Required; server assigned; patch protected	BI current-BU context

Read flow

A search or page action reaches BI Grid, which asks ProductDataSource to read `/Products`` with OData parameters. The interceptor adds the JWT. The API validates the token and Read permission, resolves BUID, filters ``IQueryable``, and only then lets OData apply filtering, paging, sorting, and count. EF Core translates the final query to SQL Server.

Write flow

BI Navigation and BI Grid collect a Product change. ProductDataSource sends POST, PATCH, or DELETE through PublicApiClient. The API validates the operation-specific permission and current BU. Successful writes update the Product table, trigger success feedback, and refresh the BU-scoped summary. Viewer writes are rejected even if issued outside the UI.

Data ownership

DOMAIN DATA	PLATFORM DATA	TRIAL AUTH DATA
Products contains the page-specific business records and validation rules.	SDK tables retain security keys, roles, role permissions, users, BU access, and shared conventions.	AppCredentials stores hashes only and links to SDK loginusers.

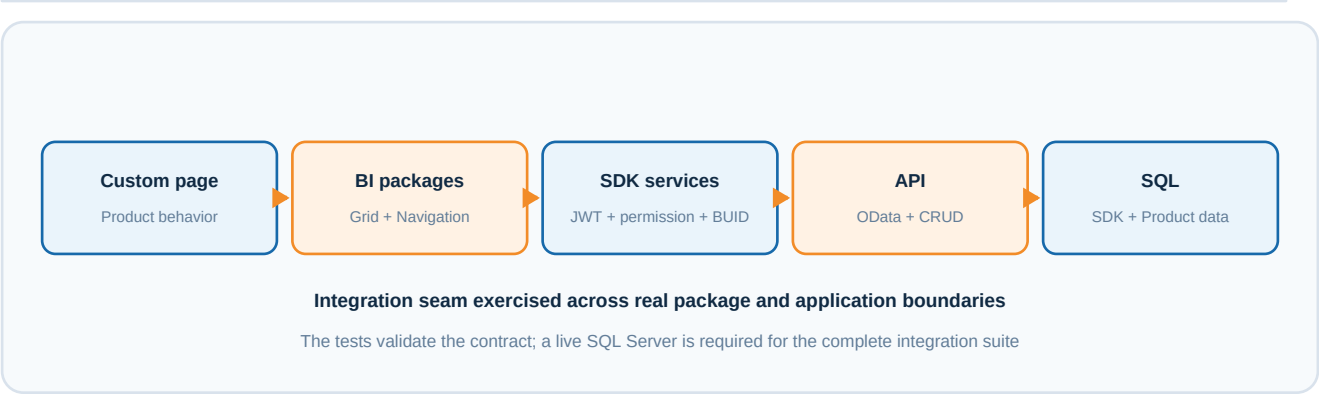
Error behavior

- Invalid login returns a generic 401 without revealing which credential failed.
- Missing or invalid tokens return 401; unauthorized protected requests clear the browser session.
- Invalid Product values return model-validation errors; absent or cross-BU keys return 404.
- Product read and write failures surface concise page messages without ending future retry behavior.

SECTION 07

Verification evidence

Checks cover the modular seam from custom page code through BI packages, SDK services, API behavior, and persistence.



Verification layer	Evidence	Status in this environment
Angular unit tests	Session, interceptor, Product data source, CRUD URLs, OData, summaries, role-aware UI, search, errors	44 tests passed across 7 files
Backend non-database tests	SDK context inheritance, registration, model mapping, JWT validation, permission declarations, schema parsing	11 tests passed
SQL integration suite	Real CRUD, validation, login, Admin/Viewer security, BUID isolation, summary isolation, idempotent schema	22 tests available; skipped because no disposable SQL Server test connection was configured
Shipped-package check	BI Navigation connected to BI Grid; Product headers and Viewer read-only behavior	Dedicated real-package test exists
Manual scenario	Anonymous state, invalid login, Admin C100 CRUD, Viewer C200 read-only, direct write rejection, cross-BU 404, logout	Documented verification flow

Interpretation

The executed baseline supports the architecture and frontend behavior without claiming a complete integration pass. The SQL-dependent tests intentionally skip when the required master connection is absent and never fall back to SQLite. A complete release check must run them against an isolated SQL Server database.

Why this matters for modularity: the verification chain crosses the boundary between page-owned code and reused BI/SalesBuzz capabilities instead of testing the Product page in isolation.

SECTION 08

Limitations and production path

The trial validates the reusable boundary; a productized extension model must formalize how pages enter SalesBuzz.

Demonstrated: a separate business page can reuse the platform contract. Not yet demonstrated: automated packaging, registration, loading, and governance inside the production SalesBuzz shell.

Current trial limitations

- The Angular page and API run as a standalone demonstration rather than as a dynamically loaded SalesBuzz page.
- The trial uses two local accounts and a small credential seeder; it is not a general identity-management system.
- Registration, password reset, refresh tokens, session revocation, account administration, and enterprise secret management are excluded.
- Local SQL Server through amd64 emulation is not a Microsoft-supported configuration on the Apple silicon development machine.
- Monitoring, audit operations, deployment ownership, service levels, disaster recovery, and production support processes are not defined.

Production extension capabilities

Capability to formalize	Purpose
Page package and manifest	Declare page identity, routes, required permissions, assets, SDK version, and dependencies.
Registration and navigation	Add approved pages to the SalesBuzz shell and navigation without editing core product code.
Loading and isolation	Load page bundles safely, define lifecycle boundaries, and prevent one page from destabilizing others.
Compatibility governance	Set supported SDK/component versions, upgrade policy, deprecation rules, and contract tests.
Security and release governance	Sign packages, review permission surfaces, manage secrets, and control deployment promotion.
Observability and ownership	Define logs, metrics, alerts, audit behavior, support ownership, and recovery procedures.

Recommended next validation

Embed one small second page into a controlled SalesBuzz shell prototype using a manifest and navigation registration contract. Reuse the same BI components, PublicApiClient, permission key pattern, and current-BU context. This would test the remaining packaging and host-integration boundary without expanding the domain scope.

SECTION 09

Conclusion

The Product page is both a working trial and a reference pattern for modular SalesBuzz extension.

BI can retain the reusable platform contract while a customer or implementation team focuses on a separate business page.

The proof of concept demonstrates a credible division of responsibility. Product-specific code owns the domain model, workflow, columns, summaries, and messages. BI and SalesBuzz provide the reusable frontend components, public client abstraction, authentication claims, permission enforcement, current-business-unit context, OData conventions, EF Core base context, and SQL security schema.

Because the integration is exercised across the frontend, API, security services, business-unit filtering, and persistence, the result is stronger than a visual mock-up. It shows that the modular page concept can be implemented. At the same time, the report does not claim that production plug-in loading already exists; packaging, registration, navigation, compatibility, governance, and support remain the next platform step.

Internship learning outcomes

- Integrated supplied frontend and backend packages into a domain-specific application.
- Separated page responsibilities from reusable platform responsibilities.
- Applied backend authorization and tenant-style business-unit isolation rather than relying on UI controls.
- Implemented OData CRUD, validation, summaries, error behavior, and SQL Server mapping.
- Used automated and manual checks to validate behavior and communicate environment-dependent coverage honestly.
- Identified the additional contracts required to turn a proof of concept into a governed extension platform.

Compact technical appendix

Item	Implemented value
Frontend	Angular 20; BI Grid; BI Navigation; PublicApiClient
Backend	ASP.NET Core / .NET 10; BI-SDK 0.2.0; OData; EF Core
Primary endpoints	POST /Auth/Login; GET /ProductSummary; OData /Products
JWT claims	Name, Role, BUID, JTI; 15-minute expiry
Product permissions	Read, Create, Update, Delete under the Products key
Data scope	C100 Admin full CRUD; C200 Viewer read-only; server-enforced BUID
Local runtime	Angular :4200; API :5201; optional SQL Server 2022 Docker service

Repository sources reviewed: project README; approved SDK permissions/current-BU design; API startup, controllers, entity and database context; SQL schema; Angular component, services, data source and column configuration; frontend and backend tests.